

Artifact Description Appendix: MIST — A Co-Design Framework for Heterogeneous, Multi-Stage LLM Inference

Supercomputing 2026

PART 1: OVERVIEW OF CONTRIBUTIONS & ARTIFACTS

A. Paper’s Main Contributions

The paper makes the following contributions for which computational artifacts are provided:

- 1) **End-to-end LLM inference simulation framework (MIST)**. MIST is an event-driven simulator that models the full serving stack — workload, system/software, and hardware layers — for heterogeneous, multi-stage LLM inference pipelines.
- 2) **Heterogeneous hardware design-space exploration (DSE)**. Using MIST, we systematically search multi-vendor prefill-decode (PD) disaggregated deployment configurations and demonstrate that heterogeneous configurations achieve higher generated tokens per second per dollar than homogeneous single-vendor baselines.
- 3) **KV cache storage co-design**. MIST enables hardware co-design for KV cache storage by evaluating five distinct memory-hierarchy architectures, revealing how storage granularity and bandwidth interact with end-to-end tail latency under varying KV reuse patterns.

B. Computational Artifacts

TABLE I
COMPUTATIONAL ARTIFACTS OVERVIEW

Artifact	Description	Related Contribution	Reproduced Elements
MIST Simulator (A1)	The complete, open-source MIST simulation engine. Accepts request traces, model specifications, and hardware parameters; returns per-request and aggregate latency, throughput, and cost metrics.	C1, C2, C3	All experiments
Reproduction Scripts (A2)	Python scripts that drive MIST, sweep the relevant parameter spaces, collect raw results, and generate publication-ready figures.	C1, C2, C3	Fig. 4 (a–d), Fig. 6 (a–b), Fig. 7 (a–b), Fig. 8, Fig. 10

A1 — MIST simulator:

<https://github.com/MIST-Simulator/MIST>

A2 — Reproduction scripts:

https://github.com/MIST-Simulator/SC_Paper_Charts

Both are released under the MIT license. A2 declares A1 as a pinned dependency in `requirements.txt`, so installing A2 installs the exact MIST revision used to produce the paper’s figures.

PART 2: ARTIFACT IDENTIFICATION

A. Relation to Contributions

The primary artifact is the MIST simulator itself, released in full. It accepts:

- **Workload input:** real or synthetic request traces (arrival times, prompt/decode token lengths, and optional prefix-KV context lengths).
- **Model specification:** model name and tensor-parallelism degree.
- **Hardware configuration:** accelerator SKU, cluster topology, and memory-hierarchy parameters.

MIST returns per-request and aggregate metrics as described in §III.G, including Time to First Token (TTFT), Time per Output Token (TPOT), Time to Last Token (TTLT), step-level runtimes, and simulation wall-clock time.

The reproduction scripts (A2) drive MIST across all experimental configurations described in the paper, write raw per-configuration result files, and invoke a separate set of plotting scripts to regenerate the figures.

a) Reproducibility of simulated data.: Every MIST-derived data point in every figure is produced by running the simulator from the scripts in A2; no simulated value is hand-transcribed. Request traces are generated or replayed under fixed random seeds and MIST’s hardware models are deterministic, so re-running a task reproduces its result CSV, and therefore its figure, exactly.

b) Vendored baseline data.: Measurements that cannot be regenerated without the original hardware or third-party simulators ship as data files in A2, each documented with its provenance: measured vLLM runs (L40S×2, H100×8, TPUv6e), Vidur outputs, LLMservingSim 2.0 outputs, and `fiio` block-device latencies. These are the reference points MIST is validated *against*, not MIST outputs.

B. Expected Results

c) Validation (Fig. 4, Fig. 6).: MIST reproduces measured on-hardware end-to-end latency distributions with a mean absolute error of 6.5–11.5% across the three validation platforms, while simulating 40–290× faster than the baseline simulators (Fig. 4(d)). For the per-step benchmarks (Fig. 6(a)), under a held-out evaluation split MIST’s ML-based LLM cluster model attains 2.1% (prefill), 0.5% (decode), and 2.1% (chunked prefill) mean absolute error, versus 18.3%, 7.7%, and 14.9% for Vidur. For KV retrieval (Fig. 6(b)), MIST tracks `fiio` measurements to 7.7% (NVMe

SSD) and 14.4% (DDR4) for block sizes at or above 16 MB; relative error is larger below 4 MB, where absolute latencies are small. These results substantiate Contribution 1.

d) *Heterogeneous DSE (Fig. 7, Fig. 8).*: The search confirms that heterogeneous multi-vendor PD-disaggregated configurations Pareto-dominate homogeneous single-vendor alternatives on the generated tokens/\$ axis, for both the chat and the code-generation use cases. Absolute tokens/\$ depends on the cloud rental prices listed in §5.1 (sampled November 2025); the performance ranking depends only on the hardware models and is unaffected by price movement. These results substantiate Contribution 2.

e) *KV Cache Storage DSE (Fig. 10).*: The five storage architectures exhibit clearly differentiated latency CDFs. Dedicated per-client memory (Case A) and platform-level shared memory (Case B) deliver the lowest tail latency for private KV caches at both context lengths, while rack-level shared SSD (Case C) is best for shared global caches with long KV sequences. These results substantiate Contribution 3.

C. Reproduction Time

Times below are measured on a 2023-class laptop-grade workstation (10 CPU cores, 32 GB RAM), not estimated.

TABLE II
MEASURED REPRODUCTION TIMES

Phase	Time
Artifact Setup	~15 min (environment + trace download)
Figure regeneration	<1 min (from shipped reference results)
T1 execution	~1 min
T2 execution	~1 min
T4 execution	~22 min
T3 execution	6–8 h (<code>--fast-eval</code>); 20+ h (full)
Total	~7–9 h (reduced T3); 21+ h (full)

A reviewer who only wishes to confirm that the figures follow from the recorded results can regenerate all seven figures in under a minute with `bash run_all.sh --plot-only`; the shipped reference results are themselves the output of the `run_T*.py` invocations recorded in `results/reference/PROVENANCE.md`.

D. Artifact Setup

Hardware: A standard Linux or macOS workstation with 8–16 CPU cores and at least 16 GB of RAM. No GPU, accelerator, or special interconnect is required: MIST is an analytical/event-driven simulator and runs entirely on CPU. An internet connection is needed to install dependencies and download the input traces.

TABLE III
REQUIRED SOFTWARE

Package	Version	Purpose	URL
Conda / Miniconda	>23.x	Environment management	https://docs.conda.io
Python	3.10–3.12	Experiment and plotting scripts	https://www.python.org
MIST	pinned in <code>requirements.txt</code>	Simulation engine	See A1 above
GenZ	pinned in <code>requirements.txt</code>	Analytical hardware model used by MIST	https://github.com/abhibhambhaniya/GenZ-LLM-Analyzer

Software: All remaining Python dependencies (numpy, pandas, scipy, scikit-learn, matplotlib, seaborn) are pinned in `requirements.txt` and installed by the setup script.

Datasets / Input:

- 1) **Microsoft Azure LLM Inference Traces (2023).** Public request traces used by the KV cache storage study. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md>
- 2) **ShareGPT conversation dataset.** Used by the end-to-end validation experiments (T1).

`scripts/download_traces.sh` fetches both and places them in the expected directory structure. The derived per-use-case traces required by T3 are committed to A2 and need no download. The measured baseline data (vLLM, Vidur, LLMservingSim 2.0, `fio`) is likewise committed to A2.

Installation & Deployment:

- 1) Clone A2: `git clone https://github.com/MIST-Simulator/SC_Paper_Chart`
- 2) Run `./setup.sh` to create the Conda environment and install all dependencies, including MIST itself.
- 3) Activate it: `conda activate mist-sc26`.
- 4) Download the traces: `bash scripts/download_traces.sh`.
- 5) Verify the installation: `python scripts/smoke_test.py` (imports MIST, runs a short simulation, and renders a figure; completes in under a minute).

E. Artifact Evaluation

The workflow consists of four independent task groups, which may be run sequentially or concurrently (`bash run_all.sh --parallel`).

T1 — End-to-End Runtime Validation (*~1 min — Reproduces: Fig. 4 a–d*)

- **T1.1** — Qwen3-32B on L40S×2 (TP2). Reproduces Fig. 4(a) and Fig. 4(d).
- **T1.2** — Llama3-70B on H100×8 (TP8). Reproduces Fig. 4(b) and Fig. 4(d).
- **T1.3** — Llama-3.1-8B on TPuv6e. Reproduces Fig. 4(c) and Fig. 4(d).

Each replays the same fixed 100-request ShareGPT trace, at the same arrival schedule, that the vLLM/Vidur/LLMservingSim baselines were measured against; holding the workload fixed is what makes the comparison controlled. `--resample-sharegpt` draws a fresh seeded sample instead, for sensitivity analysis.

Dependencies: none.

T2 — Individual Step Validation (*~1 min — Reproduces: Fig. 6 a–b*)

- **T2.1** — Per-step prefill/decode/chunked runtimes for Llama-2-70B on H100 (TP4 and TP8), evaluated on a held-out split. Reproduces Fig. 6(a).

- **T2.2** — KV cache retrieval latency for NVMe SSD and DDR4 across block sizes from 256 KB to 1 GB, against `fiio` measurements. Reproduces Fig. 6(b).

Dependencies: none.

T3 — Heterogeneous Design-Space Search

(6–8 h reduced; 20+ h full — Reproduces: Fig. 7 a–b, Fig. 8)

- **T3.1** — DSE for the chat conversation use case (no prefix-KV reuse), at 100 RPS with a TTFT P99 SLO. Reproduces Fig. 7(a) and the chat column of Fig. 8.
- **T3.2** — DSE for the code generation use case with high prefix-KV reuse, at 100 RPS with a relaxed TTFT P99 SLO. Reproduces Fig. 7(b) and the code-generation column of Fig. 8.

Both search hardware SKU $\times$ tensor parallelism $\times$ batching strategy $\times$ prefill-to-decode client ratio, and rank SLO-satisfying configurations by generated tokens/second/\$.

Reduced search-space option: `--fast-eval` restricts the search to a deterministic, stratified ~ 150 -configuration subset per use case that spans the Pareto frontier. The sweep is resumable: re-running after an interruption skips completed configurations.

Dependencies: none; T3.1 and T3.2 are independent.

T4 — KV Cache Storage Design-Space Search (~22 min — Reproduces: Fig. 10)

Evaluates the five KV cache storage architectures of Table 2 on a 256-GPU system (128 clients, H100:TP2, 4 racks), sweeping short (4K token) and long (24K token) KV contexts for both private and shared cache scenarios.

Dependencies: none.

F. Artifact Analysis

Each task group follows a two-script pipeline:

- 1) **Experiment script** (`run_{T1|T2|T3|T4}.py`): drives MIST over the configured parameter sweep, collects raw per-request and aggregate metrics, and writes structured `.csv` files to `results/<task>/`.
- 2) **Plotting script** (`plot_{T1|T2|T3|T4}.py`): reads those CSVs, applies post-processing (CDFs, normalised tokens/\$, overlaid real-hardware reference points), and writes `.pdf` figures to `figures/`. The plotting scripts do not import MIST, so figures can be regenerated without the simulator installed.

Task dependency graph:

T1.1, T1.2, T1.3 $\rightarrow$ `plot_T1.py` $\rightarrow$ Fig. 4

T2.1, T2.2 $\rightarrow$ `plot_T2.py` $\rightarrow$ Fig. 6

T3.1, T3.2 $\rightarrow$ `plot_T3.py` $\rightarrow$ Fig. 7, Fig. 8

T4 $\rightarrow$ `plot_T4.py` $\rightarrow$ Fig. 10

`run_all.sh` executes all four experiment scripts (sequentially, or concurrently with `--parallel`) and then all four plotting scripts. `run_all.sh --plot-only` regenerates every figure from existing results without re-simulating.